

Context, building, and learning with AI

Context, communication, checking, and building for your learning.

Week 6 · Learn, Vibe, Build · October 5, 2026

What you've been building

- A startup proof of concept, built in about eight hours.
- A Three.js pop-up storybook, live on Vercel.
- A planet-formation simulator grounded in a course textbook.
- A Canvas browser extension, planned by letting the AI ask 20+ questions first.
- A household assistant, with 3D-printed parts on order.
- A music visualizer where one AI writes prompts for another.

What you've been running into

- Getting the AI to build what you actually pictured.
- Links that work for you but not for anyone else.
- Running out of free credits mid-build.
- Very different starting points—and that is fine.
- Tonight: the foundations behind each of these.

Markdown: readable, portable text

- A .md file is a plain-text file—not a special app or database.
- A little punctuation adds structure: headings, lists, links, code.
- Read it in an editor, render it as a page, or paste it into your AI.

Markdown you can read and write

```
# Project notes
## What I want to learn
- Explain commits before changing files.
[Course
lesson](https://cu.learnvibe.build/lessons/w
eek-6/)
Use `git status` to see what changed.
```

Headings, lists, links,
and code—with plain
text underneath.

Context: what the AI can actually see

- Its answer depends on the messages, files and tool results available now.
- It does not automatically know your audience, preferences or project history.
- “Improve my site” is weaker than a clear audience, goal and example.

Save project context instead of repeating it

- Write down what the project is, who it is for, and what should stay true.
- Keep it short, correct and editable; update it when the project changes.
- Example: "For nervous first-time visitors. Plain HTML/CSS. Do not invent event dates."

A context file loads every session

- Coding agents such as Claude Code, Codex and Cursor can read AGENTS.md automatically.
- Your project context is there at the start of every session—no retyping.
- In a chat app, paste the context or add it to a Project.

An index points to deeper context

- A small index says which files exist and what each is for.
- Example: docs/facts.md → verified facts; docs/design.md → visual direction.
- The AI can open the relevant document instead of receiving one giant prompt.

When the result isn't what you meant

- Say what you expected and what you actually see:
“Clicking the card should open the project; nothing happens.”
- Show it: a screenshot or a site you like says more than
“make it better”.
- Ask for one change at a time, then look again.

Check the experience—not just the code

- Open it on a phone. Click every important button and link.
- Try empty input, a strange answer, or a first-time visitor's path.
- For a learning tool, check its facts and quiz answers against a trustworthy source.

Folder, repository, website: different things

- Local folder: files on your own computer.
- Git repository: those files plus a recorded history of versions.
- Deployment: a published version that other people can open.

Commit, push, deploy: three actions

- Commit saves a version and a message in your local Git history.
- Push sends your commits to the repository on GitHub.
- Deploy publishes the project through a host such as GitHub Pages or Vercel.

Your link must work on someone else's phone

- `file:///...` and `localhost:8000` point to your own computer—not a public site.
- A GitHub repository URL shows source; it may not be the live app URL.
- Use the deployed HTTPS URL; test it on another device or in a private window.

Getting it online, your way

- Ask the AI tool you already use: “Help me get this online with the tools I have. Explain each step as we go.”
- It will save a version, send it to GitHub, and publish it with a host.
- Done means a public link that opens on someone else’s phone.

APIs and MCP: how AI connects to other tools

- API: how one program requests information or actions from another.
- MCP: a standard way for AI tools to connect to other services and data.
- In Claude, these MCP connections are called connectors.

Use AI to learn about AI

- Ask for an explanation, an example, and a question to check your understanding.
- Say what you already know: “I know HTML; Git is new to me.”
- Compare its explanation with your source—not just how confident it sounds.

The reading: your AI as a coach

- Arielle Shipper · “How to Get Better at AI by Asking AI” · Every
- Ask where you are on the Eight Levels of AI Adoption, with evidence from your work.
- Ask for one next step, then: “Teach me that and help me practise.”

No long chat history? Bring your own evidence

- Paste your last two submissions or describe a recent project.
- Ask: "What can I do already? Show examples. What should I learn next?"
- Treat the AI's assessment as a suggestion; correct its assumptions.

Bring this lesson into your own AI

- Open `cu.learnvibe.build` → Everything for Week 6.
- Click “Copy lesson as markdown”; paste it into Claude or another AI.
- Ask: “Teach me this and quiz me”—then ask follow-ups in your own words.

After the slides: one conversation with Claude

- Aaron will paste the lesson and talk through it with Claude.
- The conversation may lead to building a little thing.
- Watch how questions, context and checking shape the result.

Optional in-class small-group building

- Choose a topic and your own approach; building/sharing is optional.
- Make a quiz, interactive explainer, practice tool—or an idea of your own.
- Use your AI; try the first version yourselves; check it and revise.

How to start—and what to share

- Give your AI the lesson, your learning question, or both.
- Ask for a small first version. Test whether it actually helps you learn.
- Show what you made, what you learned, and one thing you checked or changed.

This week: ONE PAGE about your learning

- What you've learned, are learning, and want to learn.
- What you want to build; emerging final-project ideas.
- Building/sharing is welcome but OPTIONAL—not required.
- Your reflections will shape consistent peer pods for at least three weeks.